C# & .NET Exam Paper 2: Collections, Memory & File I/O

Comprehensive Test covering Arrays, Lists, Stack vs Heap, and System.IO File Operations

Duration: *90 Minutes* | ***Total Marks:*** *70 Marks* | ***Format:*** *Written & Practical Coding*

Section A: Arrays & List Collections (20 Marks)

Assess data structures including fixed-size arrays and dynamic List collections.

Q1. What happens if you attempt to access numbers[5] on an array defined as int[] numbers = new int[5];? [4 Marks]

A) It returns 0

B) It automatically expands the array

C) Throws an IndexOutOfRangeException at runtime

D) Throws a compilation error

Q2. Compare C# fixed-size arrays (int[]) with dynamic generic lists (List). Highlight differences in size, methods, and syntax. [6 Marks]

Q3. Write a C# method that accepts a List of student test scores and returns the average score. If the list is empty, return 0.0. [10 Marks]

Section B: Memory Architecture — Stack vs. Heap (20 Marks)

Examines execution frames, memory allocation, value types vs reference types.

Q4. Explain how memory allocation differs when executing int x = 10; versus int[] arr = new int[10];. [10 Marks]

Q5. Analyze the following code. What are the values of listA[0] and listB[0] at the end of execution? Explain why. [10 Marks]

```
List<string> listA = new List<string> { "Apple" };
List<string> listB = listA;
listB[0] = "Banana";
Console.WriteLine(listA[0]);
```

Section C: File Handling & Input/Output (30 Marks)

Focuses on working with text files using the System.IO namespace.

Q6. Identify two advantages of using StreamReader and StreamWriter over File.ReadAllText and File.WriteAllText. [6 Marks]

Q7. Write a complete C# program that reads names from "input_students.txt" line by line, converts each name to UPPERCASE, and writes the uppercase names to "output_students.txt". Include basic exception handling using try-catch. [24 Marks]

EXAM ANSWER KEY & MARKING SCHEME

Comprehensive Solutions & Grading Criteria

Solutions: Section A: Arrays & List Collections (20 Marks)

Q1. Solution & Criteria:

Correct Answer: C. C# arrays are zero-indexed (valid indices 0 to 4). Accessing index 5 triggers an IndexOutOfRangeException.

Q2. Solution & Criteria:

Arrays have a fixed size declared at creation and use .Length. List<T> is dynamic in size, provides methods like .Add(), .Remove(), .Contains(), and uses .Count to inspect item quantity.

Q3. Solution & Criteria:

```
public static double CalculateAverage(List<double> scores) {
if (scores == null || scores.Count == 0) return 0.0;
double sum = 0;
foreach (double score in scores) {
sum += score;
}
return sum / scores.Count;
}
```

Solutions: Section B: Memory Architecture — Stack vs. Heap (20 Marks)

Q4. Solution & Criteria:

int x = 10 allocates a value type variable directly on the stack frame. int[] arr allocates the reference variable arr on the stack, which points to the array object instantiated on the heap.

Q5. Solution & Criteria:

Output: "Banana"
Explanation: List<T> is a reference type. Assigning listB = listA copies the reference pointer, not the object. Both variables point to the same memory location on the heap.

Solutions: Section C: File Handling & Input/Output (30 Marks)

Q6. Solution & Criteria:

StreamReader/Writer stream data line-by-line rather than loading entire multi-gigabyte files into RAM at once, preventing OutOfMemoryException and improving I/O throughput.

Q7. Solution & Criteria:

```
using System;
using System.IO;

class FileProcessor {
static void Main() {
try {
string[] lines = File.ReadAllLines("input_students.txt");
using (StreamWriter writer = new StreamWriter("output_students.txt")) {
foreach (string line in lines) {
writer.WriteLine(line.ToUpper());
}
}
Console.WriteLine("File processing complete.");
} catch (FileNotFoundException) {
Console.WriteLine("Error: Input file not found.");
} catch (Exception ex) {
Console.WriteLine($"Error: {ex.Message}");
}
}
}
```